A Hybrid Quantum-Classical Kernel SVM for 22-Class Crop Recommendation

23CSE463 – Quantum Computing

Vishal M.D. (Roll No. CH.SC.U4CSE24150)
Devadharshan S. (Roll No. CH.SC.U4CSE24113)

Capstone Review

October 2026

Problem Statement

- **What is the problem?**
  Given 7 field measurements (soil N, P, K, temperature, humidity, pH, rainfall), recommend the most suitable crop out of **22 classes**. On the quantum side: can a quantum machine-learning model solve such a real multi-class tabular task reliably?
- **Why does it need to be solved?**
  - Crop advice is often generic, while soil and climate differ field by field; a wrong crop wastes fertilizer, water and income.
  - Trainable quantum circuits (VQC) are hard to optimize: our own 5-qubit VQC reached only **32.1%** test accuracy.
- **Significance of solving it:**
  - Supports data-driven, precision agriculture.
  - Shows how far quantum *kernel* methods can go on real data, and honestly compares them with classical models.

Objectives

- **How we plan to solve it:** A hybrid pipeline: a fixed quantum circuit provides a similarity measure (*quantum kernel*) and a classical Support Vector Machine makes the decision. No quantum parameters need training.
- **Approach followed:**
  1. Preprocess and scale the 2,200-sample dataset (no train–test leakage).
  2. Encode each sample into a 7-qubit quantum state.
  3. Compute the fidelity kernel matrix *efficiently* (one simulation per sample).
  4. Prevent overfitting with a bandwidth parameter γ and nested cross-validation.
  5. Benchmark against tuned classical models with a significance test.
- **Algorithm used: Quantum Kernel SVM (QSVC)** with a **ZZFeatureMap** and a fidelity quantum kernel.

Proposed Algorithm (1/2): Quantum Kernel SVM (QSVC)

Introduction. QSVC replaces the kernel of a classical SVM by $K(x, x') = |\langle\phi(x)|\phi(x')\rangle|^2$, the overlap (fidelity) of two quantum states produced by a feature-map circuit. The circuit is fixed; only the classical SVM is trained, which is a convex problem.

Feature map (ZZFeatureMap):

$$|\phi(x)\rangle = \left(e^{\,i\sum_S \phi_S(x)\prod_{j\in S} Z_j}\, H^{\otimes n}\right)^r |0\rangle^{\otimes n}$$

with $n = 7$ qubits, $r = 3$ repetitions, full entanglement, and angles $\phi_i(x) = x_i$, $\phi_{ij}(x) = (\pi - x_i)(\pi - x_j)$ for inputs $x \cdot \gamma$.

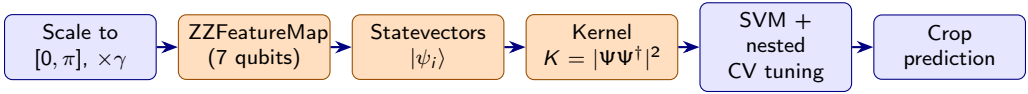
Step-by-step:

1. **Scale:** StandardScaler, then MinMax to $[0, \pi]$; multiply by γ.
2. **Encode:** simulate $|\phi(x_i)\rangle$ for every sample (7 qubits $\Rightarrow$ 128 amplitudes).
3. **Kernel:** $K_{ij} = |\langle\phi(x_i)|\phi(x_j)\rangle|^2$, with $K_{ii} = 1$.
4. **Train:** SVM on the precomputed K (one-vs-one over 22 classes).
5. **Tune:** search reps, entanglement, γ, C with nested CV and the one-standard-error rule.
6. **Predict:** compute the kernel of a new sample against the training states, then classify.

Proposed Algorithm (2/2): Pseudocode and Flowchart

```
Algorithm QSVC-Train(X_train, y_train, Grid)                // Output: model, best config
 1  Fit StandardScaler + MinMaxScaler[0, pi] on X_train; transform X_train
 2  for each (reps, ent, gamma) in Grid:
 3      FM <- ZZFeatureMap(n = 7, reps, ent)
 4      for i = 1 .. N:  |psi_i> <- Statevector( FM(gamma * x_i) )   // 1 circuit / sample
 5      Psi <- stack(|psi_1>, ..., |psi_N>)                 // N x 2^7
 6      K <- |Psi . Psi^dagger|^2 ;  K_ii <- 1              // fidelity kernel
 7      for each C in Grid:  score <- nested stratified CV of SVC(K, C)
 8  (reps*, ent*, gamma*, C*) <- best score (simplest within 1 standard error)
 9  return SVC(kernel = precomputed, C*).fit(K*, y_train)

Algorithm QSVC-Predict(x_new)
10  |psi_new> <- Statevector( FM*(gamma* . scale(x_new)) );  k <- |Psi* . |psi_new>|^2
11  return model.predict(k)
```



Orange = quantum part (simulated), blue = classical.

Algorithm Complexity

Stage	Time	Space
Encode one sample (statevector)	$O(r\,n^2\,2^n)$	$O(2^n)$
Encode all N samples	$O(N\,r\,n^2\,2^n)$	$O(N\,2^n)$
Kernel matrix K $(N \times N)$	$O(N^2\,2^n)$	$O(N^2)$
SVM training (one-vs-one, SMO)	$O(N^2)$ to $O(N^3)$	$O(N^2)$
Predict M new samples	$O(M\,N\,2^n)$	$O(M\,N)$

n = qubits (7), r = repetitions (3), N = training samples (1,760), M = test samples (440).

- **Why:** a ZZ feature map has $O(r\,n^2)$ gates (full entanglement: $\binom{n}{2}$ pairs per repetition); each gate touches a 2^n-amplitude state.
- **Our speed-up:** the naive method runs one circuit per *pair*: $O(N^2\,r\,n^2\,2^n)$. We simulate each sample once and use one matrix product, removing the $r\,n^2$ factor per kernel entry.
- **Memory:** states $\approx$ 3.6 MB, training kernel $\approx$ 24.8 MB, test kernel $\approx$ 6.2 MB.
- The 2^n factor comes from *classical simulation*; on hardware each of the N^2 entries needs $O(1/\epsilon^2)$ shots instead. Model selection (270 configurations) reuses kernels cached per (r, ent, γ): 54 kernels.

Implementation

- **Language:** Python 3
- **Libraries:** FastAPI (REST backend), Next.js 16 / React 19 (TypeScript, Tailwind CSS), Qiskit 1.1.0, scikit-learn
- **Explanation:**
  - `data_prep.py`: split and scale (scalers fitted on training data only)
  - `qkernel.py`: feature map, statevector simulation, fidelity kernel
  - `trainv2.py`: 270-configuration nested-CV search, final SVM
  - `baselines.py`, `analysis.py`: benchmarking and error analysis
  - api.py: FastAPI REST endpoint; web/: Next.js multi-lingual UI (Vercel); predict.py: CLI inference

Core code

```
fm = ZZFeatureMap(7, reps=3,
                  entanglement="full")
# one simulation per sample
S = compute_states(fm, X_train*gamma)
# fidelity kernel |<a|b>|^2
K = np.abs(S.conj() @ S.T) ** 2
np.fill_diagonal(K, 1.0)

svc = SVC(kernel="precomputed", C=10)
svc.fit(K, y_train)
```

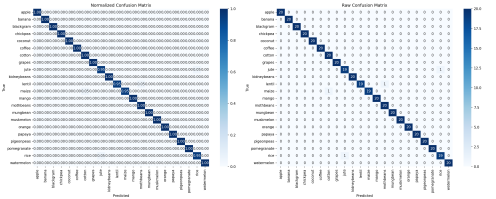
Results and Analysis

Input: X_{train} 1760×7, X_{test} 440×7 (scaled to $[0, \pi]$), 22 crop labels; reps=3, full entanglement, $\gamma = 0.1$, $C = 10$.

Output: predicted crop per sample, kernel matrices (1760×1760, 440×1760), metrics.

Metric	Value
Test accuracy	**99.09%** (4/440 errors)
Macro-F1	0.9909
Nested CV accuracy	$98.20\% \pm 0.82\%$
Train accuracy	99.55%

Model	Test acc.	Errors
Random Forest	99.55%	2
QSVC (ours)	**99.09%**	4
Linear SVM	99.09%	4
RBF-SVM	98.86%	5
k-NN	98.41%	7



Observations

- 17 of 22 crops: F1 = 1.00; weakest jute, rice (0.95).
- QSVC vs. Random Forest: McNemar $p = 0.617$, not significant; linear SVM ties.
- Earlier attempts: VQC 32.1%, first kernel prototype 65.0% (overfit); γ tuning and statevector kernels fixed it.

Conclusion

- **What was achieved:** A working 7-qubit quantum-kernel SVM that classifies 22 crops with **99.09%** test accuracy (macro-F1 0.9909) and a rigorous evaluation (nested CV, tuned baselines, McNemar test).
- **Was the problem addressed?** Yes. The earlier VQC reached only 32%, and the first kernel prototype overfit (65%). The final pipeline is accurate, fast and reproducible.
- **Key observations:**
  - Bandwidth γ and tuning are decisive for quantum kernels (kernel concentration).
  - Statevector simulation cut kernel time from $\sim$84 minutes to seconds.
  - Random Forest (99.55%) and linear SVM (99.09%) perform equally well; McNemar $p = 0.617$.
- **Effectiveness of the algorithm:** Effective and classical-level on this dataset, but *not* shown to beat classical methods; the dataset is easy and runs are noiseless simulations. Next: noise study, harder data, hardware run.

Thank You

Questions & Discussion